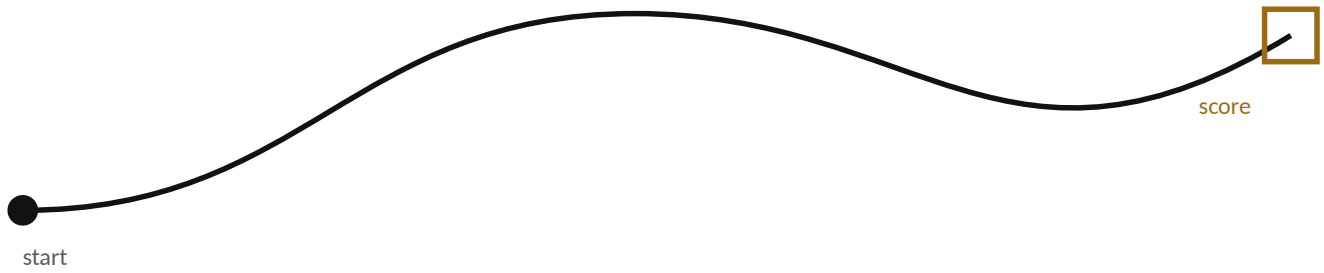
Coding FTC in Java

From no dev environment to a scoring autonomous with vision. Everything in this packet is in the order we'll cover it, and it's yours to take home.



Name _____ Team number _____ Role on team _____

Follow along, then take it home

Page numbers are called out from the front of the room. Nothing here expires when the session ends.

Three kinds of pages. Explainers cover the idea. **Code** pages are complete, working programs you can type or copy. Each one says which explainer page it belongs to. **Handouts** are pages meant to leave this packet: a script for asking your school for a computer, a worksheet to fill in after the game reveal, and a four-week plan. Tear them out, they're one-sided on purpose.

Three ways to write code, and when to leave each

All three are real. Teams have won on all three. The wrong move is waiting until you have the "right" one before you write anything.

Question	Blocks	OnBot Java	Android Studio
Do I have to install anything?	No	No	Yes
Works on a Chromebook?	Yes	Yes	No
Real Java?	No	Yes	Yes
Git / version history	No	No	Yes
Outside libraries (Pedro, NextFTC)	No	No	Yes
Debugger and refactoring tools	No	No	Yes
Time to a first working OpMode	Minutes	Minutes	An evening

Blocks

Drag-and-drop in a browser, served by the Control Hub itself. Genuinely useful past rookie year for quick hardware tests: checking a motor direction, jogging a servo to find its positions. The real limit is size: past a few hundred blocks it becomes unreadable, and there's no version history, so a mistake is permanent.

OnBot Java

Real Java, typed in a browser, compiled on the hub. No install, no laptop requirements beyond a working browser. This is a legitimate full-season answer if your school computers are locked down. What you give up: outside libraries, which means no Pedro Pathing and no command-based frameworks, and no Git.

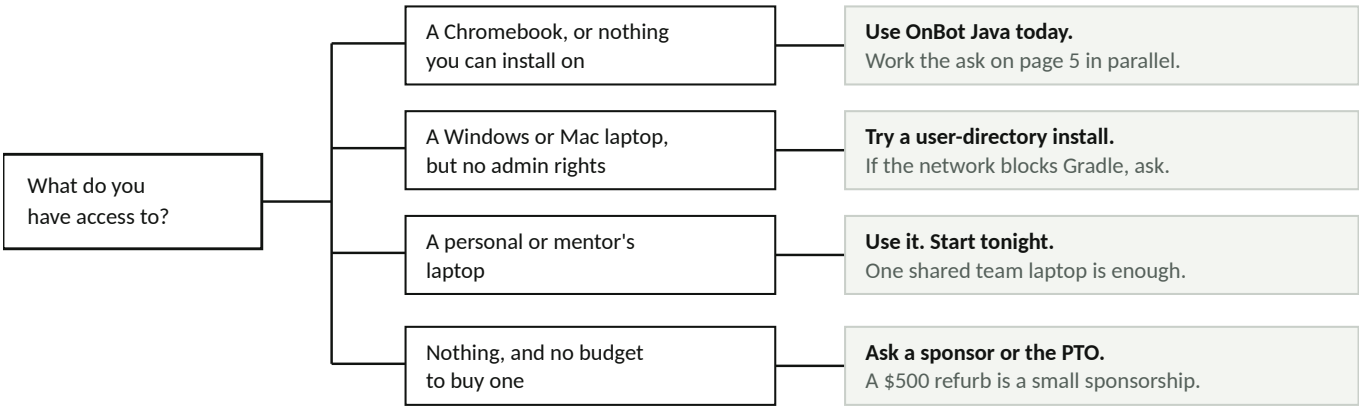
Android Studio

The full project on your own machine. You get Git, outside libraries, a real editor, and a debugger. The cost is the install, a computer that can handle it, and a first build that takes a while. Everything past page 15 in this packet requires it.

> **The honest rule.** Start where you can start today. Move up when a limit actually blocks something you're trying to do, not because someone told you the "real" teams use Android Studio. And you can mix: a driver can run a Blocks test OpMode on the same hub your programmer is deploying to from Android Studio.

Getting a computer that can run Android Studio

Most teams that never get to Android Studio didn't decide against it. They hit a locked-down school laptop and quit there. Find your situation below.



Diagnose before you argue

"The school laptop won't work" is four different problems with four different fixes. Figure out which one you have before you talk to anybody:

- **No admin rights** — you can often still install to your user folder. Try it first.
- **Installs blocked entirely** — this is a policy problem, not a technical one. Page 5.
- **It's a Chromebook** — Android Studio will not run. OnBot Java, or a different machine.
- **The network blocks the download** — the first Gradle build needs internet. Do it once at home or on a phone hotspot; after that you're mostly offline.

Minimum spec that actually works

Part	Minimum	Why
Memory	16 GB	8 GB builds, but slowly enough to hurt
Storage	256 GB SSD	The SDK, Gradle cache and Android tooling are large
Processor	64-bit x86-64 or Apple Silicon	Android Studio requirement
Ports	One working USB port	Wired deploy when Wi-Fi is bad at competition

Roughly a $500 refurbished business laptop. Not a Chromebook, not a tablet.

What to say, and who to ask

Teams lose this conversation by asking for "admin rights." Ask instead for one specific thing, with a purpose and a price attached.

Who to go to, in order

1. **Your CTE director, technology director, or principal.** They can grant an exception.
2. **Your coach or lead mentor first, always** — an adult asking lands differently, and they may already know who decides.
3. **Not the help desk.** They enforce policy, they don't write it. They will say no because no is the only answer they're allowed to give.

Ask for one of these, in this order

1. One **exempted device** — a laptop off the standard image, kept in the robotics room, with the coach as the responsible adult.
2. A **local admin exception** on a laptop the team already has.
3. Permission to use a **donated or personal laptop** on the guest network, where it never touches district systems.

Say this

"Our robotics team programs the robot in Android Studio, which is the industry-standard tool our students will use in college and in industry. It has to install to the machine, so it can't run on the standard student image.

We're asking for one exempted device for the robotics room. It stays in the building, our coach is responsible for it, and it doesn't need access to any district systems, just internet access and a USB port.

There are _____ students on the team who need it, and without it they're limited to a browser-based tool that can't use the libraries the rest of the program uses."

Bring this with you

- The spec sheet from page 4
- How many students it affects
- A date you need it by
- Who the responsible adult will be
- An offer: the team will handle setup

If the answer is no

- Ask what *would* be approvable
- Ask for the guest-network option instead
- Ask a sponsor or the PTO to fund it
- Run OnBot Java meanwhile. Don't stall.
- Ask again in January with results to show

Notes from your conversation

Who I asked

What they said

Follow up by

Installing the SDK and building it once

Do this at home, on real internet, before your next meeting. The first build is the slow one; every build after is fast.

1 — Install Android Studio

Download from `developer.android.com/studio`. Accept the default components. The FTC SDK pins a minimum version and it moves. Season releases through v11.1 needed Ladybug (2024.2); the v11.2 offseason release raised it to **Narwhal 3 Feature Drop (2025.1.3)** and lists breaking build changes. Read the README of the exact release you download before you install anything, because an older Android Studio will fail the build with an error that does not say why.

2 — Get the FTC SDK

The SDK is a GitHub project you copy and then edit. Two ways:

```
Option A — with Git (recommended, you get version history for free)
  git clone https://github.com/FIRST-Tech-Challenge/FtcRobotController.git

Option B — no Git
  Open the GitHub page, click Code, then Download ZIP, then unzip it.
```

Better still: on GitHub click **Use this template** to make your own team repository from it. Then your season's code lives in your team's account from day one.

3 — Open it and wait

In Android Studio choose **Open** and pick the folder you just made. Gradle will sync and download dependencies. On a first run this takes several minutes and looks like nothing is happening. Let it finish before you touch anything.

4 — Know where your code goes

```
FtcRobotController/     ← don't edit this. Sample OpModes live here.
    .../external/samples/     copy FROM here

TeamCode/               ← everything you write goes here
    .../teamcode/             paste INTO here
```

Copy a sample into `teamcode`, rename the class to match the filename, and edit that. Never edit the samples in place, because next season's SDK update overwrites them.

> **Set up Git today, not in January.** Commit at the end of every meeting. The single most common way FTC teams lose an entire autonomous is a laptop that dies in February with the only copy on it. A free GitHub account fixes this permanently.

If the build fails

What you see	Almost always
Gradle sync hangs or times out	Network. Use a phone hotspot and retry.
"SDK location not found"	Let Android Studio install the Android SDK when prompted.
Unsupported Gradle / JDK version	Your Android Studio is older than the FTC SDK expects. Update it.

Talking to the Control Hub

The hub makes its own Wi-Fi network. Your laptop joins it, and everything happens over that link: Blocks, OnBot Java, the config file, and deploying from Android Studio.

1 — Connect

1. Power the hub from a charged battery. Wait for the LED to settle.
2. On your laptop, join the Wi-Fi network named after the hub.
3. Browse to `192.168.43.1:8080`. That's the hub's web interface.
4. **Change the default password.** Inspectors check this, and you'll fail inspection with it unchanged.

2 — Build the configuration file

The hub doesn't know what's plugged into it. The config file is the map from physical ports to the names your code uses, and a mismatch here is the #1 cause of "my code doesn't work."

Pick a naming convention now and never change it

Port	Config name	In code
Motor 0	`frontLeft`	`hardwareMap.get(DcMotor.class, "frontLeft")`
Motor 1	`backLeft`	same spelling, same capitalization
Motor 2	`frontRight`	
Motor 3	`backRight`	

Then write those names on masking tape and stick it on the robot next to each port. Your future self at 11pm before a competition will thank you.

3 — Deploy

Android Studio: with your laptop on the hub's Wi-Fi, press Run. The app installs to the hub. **OnBot Java:** the browser builds it on the hub directly. Either way your OpMode then appears in the dropdown on the Driver Station.

The three failures you will hit this week

Symptom	Cause
Crash the instant you press INIT	Config name doesn't match the string in your code. Case matters: `frontleft` is not `frontLeft`.
Your OpMode isn't in the dropdown	Missing `@TeleOp` or `@Autonomous` above the class, or the build actually failed. Read the build output, not the phone.
One wheel spins backwards	Motor direction, not your math. Set `setDirection(REVERSE)` on that motor.

> **When something crashes mid-match,** the hub keeps a log. Download `robotControllerLog.txt` from the web interface and read the stack trace. It tells you the exact line. This is a skill worth practising on purpose.

Mecanum drive, robot-centric vs field-centric

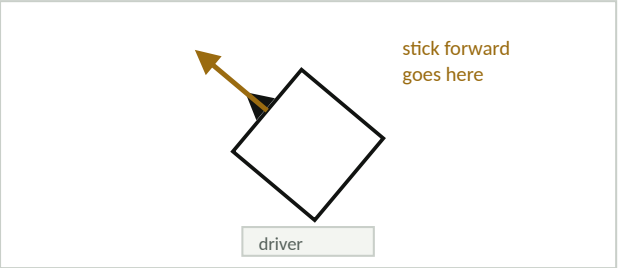
Mecanum wheels have angled rollers, so spinning them in different combinations pushes the robot sideways as well as forward. You mix three inputs into four wheel powers: forward, strafe and turn.

```
frontLeft  = drive + strafe + turn       then divide all four by the largest
backLeft   = drive - strafe + turn       absolute value if it exceeds 1.0, so
frontRight = drive - strafe - turn       the ratios stay right instead of
backRight  = drive + strafe - turn       getting clipped.
```

The one difference that matters to your driver

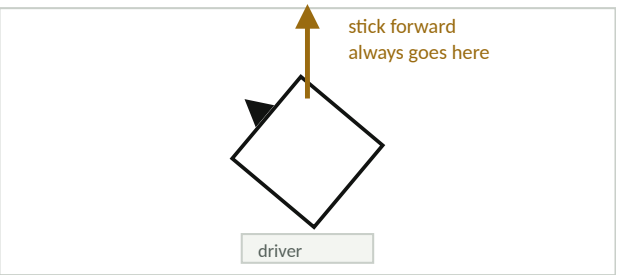
Robot-centric

Forward = wherever the nose points



Field-centric

Forward = away from the driver, always



The robot is turned the same amount in both. Only the meaning of the stic

Robot-centric

Simple and deterministic. Nothing to calibrate, nothing to drift. The catch is that when the robot faces the driver, left and right invert, and new drivers crash into things constantly.

Field-centric

You rotate the joystick vector by the robot's heading before mixing. Much easier to learn. The catch is that it depends on the IMU: if the heading drifts, or the robot started at the wrong angle, everything is wrong.

> **Do both.** Put them on a toggle and bind a "reset heading" button. Let the driver pick in practice. That's what the example on the next page does.

A complete TeleOp with both drive modes

Copy this into `TeamCode`. Change the four config names to match your configuration file and nothing else. Options button toggles field-centric; Back resets heading.

```
package org.firstinspires.ftc.teamcode;
// Imports: LinearOpMode, TeleOp, DcMotor, IMU, AngleUnit,
// RevHubOrientationOnRobot. In Android Studio press Alt+Enter
// on any red name and it adds the import for you.

@TeleOp(name = "Mecanum Drive", group = "Drive")
public class MecanumDrive extends LinearOpMode {

    @Override
    public void runOpMode() {
        DcMotor frontLeft  = hardwareMap.get(DcMotor.class, "frontLeft");
        DcMotor backLeft   = hardwareMap.get(DcMotor.class, "backLeft");
        DcMotor frontRight = hardwareMap.get(DcMotor.class, "frontRight");
        DcMotor backRight  = hardwareMap.get(DcMotor.class, "backRight");

        // Left side usually needs reversing. If a wheel spins the wrong
        // way, fix it HERE, not in the math below.
        frontLeft.setDirection(DcMotor.Direction.REVERSE);
        backLeft .setDirection(DcMotor.Direction.REVERSE);

        // Brake when the stick is released, or the robot coasts on.
        for (DcMotor m : new DcMotor[]{frontLeft, backLeft, frontRight, backRight})
            m.setZeroPowerBehavior(DcMotor.ZeroPowerBehavior.BRAKE);

        IMU imu = hardwareMap.get(IMU.class, "imu");
        imu.initialize(new IMU.Parameters(new RevHubOrientationOnRobot(
                RevHubOrientationOnRobot.LogoFacingDirection.UP,
                RevHubOrientationOnRobot.UsbFacingDirection.FORWARD)));

        boolean fieldCentric = false;
        boolean lastToggle   = false;

        waitForStart();

        while (opModeIsActive()) {

            if (gamepad1.options && !lastToggle) fieldCentric = !fieldCentric;
            lastToggle = gamepad1.options;

            if (gamepad1.back) imu.resetYaw();   // square up, then press Back

            double drive  = -gamepad1.left_stick_y;   // stick up is negative
            double strafe =  gamepad1.left_stick_x;
            double turn   =  gamepad1.right_stick_x;

            if (fieldCentric) {
                double h = imu.getRobotYawPitchRollAngles().getYaw(AngleUnit.RADIANS);
                double rotStrafe = strafe * Math.cos(-h) - drive * Math.sin(-h);
                double rotDrive  = strafe * Math.sin(-h) + drive * Math.cos(-h);
                strafe = rotStrafe;
                drive  = rotDrive;
            }

            // Keep the ratios instead of clipping them.
            double denom = Math.max(Math.abs(drive) + Math.abs(strafe)
                                  + Math.abs(turn), 1.0);

            frontLeft .setPower((drive + strafe + turn) / denom);
            backLeft  .setPower((drive - strafe + turn) / denom);
            frontRight.setPower((drive - strafe - turn) / denom);
            backRight .setPower((drive + strafe - turn) / denom);

            telemetry.addData("mode", fieldCentric ? "FIELD" : "ROBOT");
            telemetry.update();
        }
    }
}
```

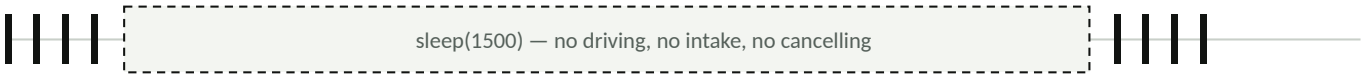
Test order: each wheel alone first, then forward, then strafe, then turn, then field-centric. Strafing being weaker than driving is normal, because the rollers waste force.

Never block the loop

This is the single most important idea in this packet. Everything about how a robot feels to drive comes down to it.

Your TeleOp runs a loop, fifty or more times a second. Every pass it reads the gamepad, decides what everything should do, and sets powers. If any code inside that loop *waits*, the whole robot waits, drivetrain included.

With sleep() — the robot is frozen



Each tick is one pass of the loop. The gap is 1.5 seconds of a match you can't drive.

With a state machine — the loop never stops

The fix, in one sentence

Instead of waiting, remember what you're doing and check every pass whether it's time to move on. That's a **state machine**: an enum of the states a mechanism can be in, a `switch` in the loop, and a condition for each transition.

When to move up to a framework

Once you have three or more mechanisms, hand-written state machines start colliding with each other. The packaged version of this pattern is called **command-based**: each mechanism becomes a subsystem, each action becomes a command, and a scheduler runs them.

- **NextFTC** and **SolversLib** are the current, maintained options for FTC.
- **FTCLib** is no longer actively maintained. You'll find old tutorials for it. Skip them.
- Both require Android Studio. Neither is worth it for a drivetrain and one intake.

> **Rule to remember:** if any code path in TeleOp can block for longer than one pass of the loop, that's a bug, even if it looks like it works on the bench.

A lift as a state machine

Press A and the lift raises, opens the claw, waits, and retracts, while the driver keeps full control of the drivetrain the entire time.

```
public class LiftTeleOp extends LinearOpMode {

    // Every state the lift can be in. Add states, not sleeps.
    enum LiftState { IDLE, RAISING, RELEASING, RETRACTING }
    static final int HIGH = 1800, TOLERANCE = 20;   // ticks — measure yours
    static final double OPEN = 0.6, CLOSED = 0.2;

    @Override
    public void runOpMode() {
        DcMotor lift  = hardwareMap.get(DcMotor.class, "lift");
        Servo   claw  = hardwareMap.get(Servo.class,   "claw");

        lift.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);
        lift.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
        LiftState  state = LiftState.IDLE;
        ElapsedTime timer = new ElapsedTime();

        waitForStart();

        while (opModeIsActive()) {

            switch (state) {

                case IDLE:
                    if (gamepad1.a) {
                        lift.setTargetPosition(HIGH);
                        lift.setMode(DcMotor.RunMode.RUN_TO_POSITION);
                        lift.setPower(0.8);
                        state = LiftState.RAISING;
                    }
                    break;

                case RAISING:
                    // Don't wait for it. Ask if it's there yet.
                    if (Math.abs(lift.getCurrentPosition() - HIGH) < TOLERANCE) {
                        claw.setPosition(OPEN);
                        timer.reset();
                        state = LiftState.RELEASING;
                    }
                    break;

                case RELEASING:
                    if (timer.seconds() > 0.3) {      // servo travel time
                        claw.setPosition(CLOSED);
                        lift.setTargetPosition(0);
                        state = LiftState.RETRACTING;
                    }
                    break;

                case RETRACTING:
                    if (Math.abs(lift.getCurrentPosition()) < TOLERANCE) {
                        lift.setPower(0);
                        state = LiftState.IDLE;
                    }
                    break;
            }

            // An escape hatch. Press B at any point to bail out.
            if (gamepad1.b) { lift.setTargetPosition(0);
                              state = LiftState.RETRACTING; }

            // ─── This runs every pass, in every state ───
            // Mecanum mixing from page 9 goes here. The driver never
            // loses the robot, no matter what the lift is doing.

            telemetry.addData("lift state", state);
            telemetry.addData("position", lift.getCurrentPosition());
            telemetry.update();
        }
    }
}
```

Why this beats `sleep()`: the driver never loses the robot, you can cancel mid-sequence, two mechanisms can run at once, and telemetry shows exactly which state it's stuck in.

Auto is free points, and the ladder to climb

Every season has a low-effort autonomous worth real points. Leave the starting position, park somewhere specific, deliver a preload. It's usually fifteen lines of code and four seconds of movement.

The pattern we see every year: a team gets the robot driving in October, feels good, and leaves the Autonomous file empty until the last league meet in December. Those are points they gave away in every single match, and auto is the one part of a match where nothing but your code decides the outcome.

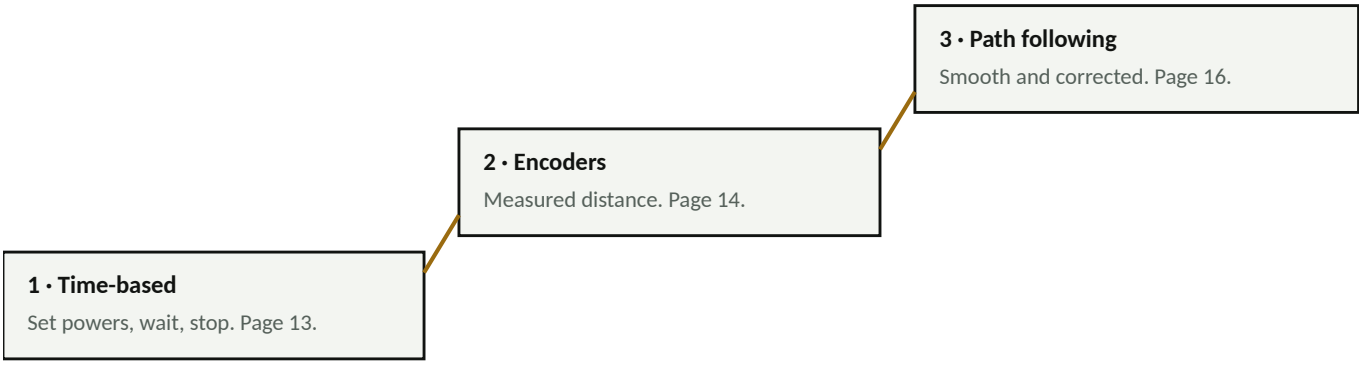
Your order of operations

1. **Get the robot driving.** That's the prerequisite, not the finish line.
2. **Write a park auto that works every time.** Run it ten times in a row. If it works ten out of ten, it's done.
3. **Add one scoring action.** One. Prove it, then stop.
4. **Then** chase the multi-cycle autos, the pathing libraries, the vision.

On ranking: auto points matter because they win close matches, and match wins are what your ranking is built on. Read Game Manual Part 1 for this season's scoring and any auto-specific bonus. That changes year to year.

The ladder

Rookies: rung 1 by week two, rung 2 by week four. Rung 3 is a January project, not a September one.



Rung	Good	Bad
Time-based	Working in ten minutes. No tuning, no extra hardware, anyone can write it. A perfectly good park auto.	Distance depends on battery voltage: a fresh battery overshoots, a tired one stops short. Carpet and wheel wear change it. It cannot correct itself.
Encoders	Distance is measured, not guessed. Repeatable across battery states. Still no outside libraries needed.	Wheel slip lies to you. Mecanum strafe distances are unreliable. Chaining many moves is slow and jerky because you stop between each one.
Pathing	Fast, smooth, accurate, corrects while moving, and you can chain a whole auto together.	Real setup and a tuning session. Wants dead-wheel odometry or a Pinpoint/OTOS sensor to localize properly, which costs money.

Diagnostic question: if your auto works on a full battery and misses on a low one, you're on rung 1 and it's time to move up.

A time-based auto that parks

This is rung one. It is a complete, legal, scoring autonomous, and you can have it working before you leave today. Tune the two numbers to your field.

```
package org.firstinspires.ftc.teamcode;

import com.qualcomm.robotcore.eventloop.opmode.Autonomous;
import com.qualcomm.robotcore.eventloop.opmode.LinearOpMode;
import com.qualcomm.robotcore.hardware.DcMotor;

@Autonomous(name = "Auto - Park (time)", group = "Auto")
public class AutoParkTime extends LinearOpMode {

    DcMotor frontLeft, backLeft, frontRight, backRight;

    @Override
    public void runOpMode() {
        frontLeft  = hardwareMap.get(DcMotor.class, "frontLeft");
        backLeft   = hardwareMap.get(DcMotor.class, "backLeft");
        frontRight = hardwareMap.get(DcMotor.class, "frontRight");
        backRight  = hardwareMap.get(DcMotor.class, "backRight");

        frontLeft.setDirection(DcMotor.Direction.REVERSE);
        backLeft .setDirection(DcMotor.Direction.REVERSE);

        telemetry.addLine("Place the robot against the wall reference. Ready.");
        telemetry.update();
        waitForStart();

        if (isStopRequested()) return;

        drive(0.4, 0, 0, 1200);   // forward, 1.2 s
        drive(0, 0.4, 0,  800);   // strafe right, 0.8 s
        stopDriving();
    }

    /** Runs the drivetrain at a power for a number of milliseconds. */
    void drive(double fwd, double strafe, double turn, long ms) {
        frontLeft .setPower(fwd + strafe + turn);
        backLeft  .setPower(fwd - strafe + turn);
        frontRight.setPower(fwd - strafe - turn);
        backRight .setPower(fwd + strafe - turn);
        sleep(ms);
        // sleep() is acceptable HERE and only here: in auto there is no
        // driver to starve. Never do this in TeleOp. See page 10.
    }

    void stopDriving() {
        frontLeft.setPower(0);  backLeft.setPower(0);
        frontRight.setPower(0); backRight.setPower(0);
    }
}
```

Make it repeatable before you make it fancy

- **Same start every time.** Build a physical alignment reference against the field wall and use it every single match. Nothing above this rung saves you from a robot placed differently each time.
- **Same battery state.** Test on a mid-charge battery, not a fresh one, because that's closer to what match four feels like.
- **Ten runs in a row.** If it works ten out of ten, it's done. If it works eight out of ten, it's not a good auto. It's a coin flip you'll lose twice a tournament.

An encoder auto with heading hold

Rung two. Distances are measured instead of timed, and the IMU holds the line. This one survives a low battery.

```
@Autonomous(name = "Auto - Park (encoders)", group = "Auto")
public class AutoParkEncoder extends LinearOpMode {

    // Measure these. Push the robot 100 cm by hand and read the encoder.
    static final double TICKS_PER_REV  = 537.7;   // goBILDA 312 RPM
    static final double WHEEL_DIAM_CM  = 9.6;
    static final double TICKS_PER_CM   = TICKS_PER_REV / (WHEEL_DIAM_CM * Math.PI);

    DcMotor fl, bl, fr, br;
    IMU imu;

    @Override
    public void runOpMode() {
        fl = hardwareMap.get(DcMotor.class, "frontLeft");
        bl = hardwareMap.get(DcMotor.class, "backLeft");
        fr = hardwareMap.get(DcMotor.class, "frontRight");
        br = hardwareMap.get(DcMotor.class, "backRight");
        fl.setDirection(DcMotor.Direction.REVERSE);
        bl.setDirection(DcMotor.Direction.REVERSE);

        imu = hardwareMap.get(IMU.class, "imu");
        imu.initialize(new IMU.Parameters(new RevHubOrientationOnRobot(
                RevHubOrientationOnRobot.LogoFacingDirection.UP,
                RevHubOrientationOnRobot.UsbFacingDirection.FORWARD)));

        waitForStart();
        if (isStopRequested()) return;

        imu.resetYaw();
        driveStraight(90, 0.5);    // 90 cm forward, holding heading
        driveStraight(-30, 0.4);
    }

    /** Drives a distance in cm while correcting heading with the IMU. */
    void driveStraight(double cm, double power) {
        int target = (int)(cm * TICKS_PER_CM);
        int start  = fl.getCurrentPosition();
        double heading0 = yaw();
        int sign = cm > 0 ? 1 : -1;

        long deadline = System.currentTimeMillis() + 6000;   // never hang

        while (opModeIsActive() && System.currentTimeMillis() < deadline
               && Math.abs(fl.getCurrentPosition() - start) < Math.abs(target)) {

            // Proportional heading correction. Raise 0.02 if it wanders,
            // lower it if it oscillates. IF IT CORRECTS THE WRONG WAY,
            // swap the two signs below. Depends on your motor directions.
            double error      = angleWrap(heading0 - yaw());
            double correction = error * 0.02;

            // Ease off near the target so it doesn't slam to a stop.
            double remaining = Math.abs(target) - Math.abs(fl.getCurrentPosition() - start);
            double scale = Math.min(1.0, Math.max(0.25, remaining / (15 * TICKS_PER_CM)));
            double p = power * scale * sign;

            fl.setPower(p - correction);  bl.setPower(p - correction);
            fr.setPower(p + correction);  br.setPower(p + correction);

            telemetry.addData("remaining cm", remaining / TICKS_PER_CM);
            telemetry.update();
        }
        fl.setPower(0); bl.setPower(0); fr.setPower(0); br.setPower(0);
    }

    double yaw() { return imu.getRobotYawPitchRollAngles().getYaw(AngleUnit.DEGREES); }

    /** Keeps an angle in -180..180, so 179 to -179 is 2 degrees, not 358. */
    double angleWrap(double deg) {
        while (deg >  180) deg -= 360;
        while (deg < -180) deg += 360;
        return deg;
    }
}
```

What changed conceptually: the loop now *watches* a sensor and reacts instead of counting down a clock. That is the same idea as page 10, which path-following libraries take further.

Plan your first auto

Do this with your whole team in the first meeting after kickoff, before anyone designs a mechanism. Twenty minutes. It sets the target for your first month.

1 What is the simplest thing that scores in auto this season?

Look for parking, leaving the start zone, or delivering something the robot already holds.

2 What does the robot have to be able to do for that?

Circle one:

Drive only · **Drive + one mechanism** · **Needs vision**

If you circled "needs vision," find a second option that doesn't. You want something scoring before vision exists.

3 Can we do it with time-based moves alone? Y / N

If yes, that's your week-two target and you don't need anything past page 13. If no, what's the one thing that makes it not work?

4 Dates

Robot drives in TeleOp by _____

Park auto running ten-for-ten by _____

One scoring action added by _____

Who owns the auto code _____

5 Points on the table

Our simplest auto is worth _____ points. Across a _____-match qualifier that's _____ points we either score or give away.

Write that number somewhere your team will see it.

Pedro Pathing, and why we recommend it

Rung three. Instead of a list of moves, you describe a curve across the field and a library drives it continuously, without stopping between segments.

The two real options

	Road Runner	Pedro Pathing
Approach	Plans a motion profile in advance and follows it	Continuously steers back toward the path as it goes
Maturity	Older, deeply proven, very widely used	Newer, now the most common choice in FTC
Tuning	Longer and more mathematical	Shorter, about one meeting
Getting bumped	Recovers, but is replaying a plan	Recovers naturally, because correcting is what it does
Docs	Thorough, written for engineers	Written for teams, with an active Discord

Why Pedro for most teams

- **It corrects instead of replaying.** Getting bumped in auto is the actual failure mode in matches, and Pedro handles it as a normal condition rather than an exception.
- **Tuning is short enough to finish in one meeting.** That matters more than it sounds. A tuning process a team can't finish is a library the team doesn't use.
- **The visualizer.** Draw the path on a field diagram in your browser, drag the control points until it looks right, copy the generated code out. You can plan an auto without a robot.
- **Panels** gives you the robot's live position in a browser while it drives, which turns "it went the wrong way" into something you can actually debug.

> **Don't switch mid-season.** If Road Runner is already working on your robot, keep it. The library you know beats the library that's better.

The prerequisite nobody mentions: localization

Path following only works if the robot knows where it is. That's localization, and your options run from free to good:

Option	Cost	Accuracy
Drive motor encoders	Free (you already have them)	Workable, but drifts on mecanum because the wheels slip
Dead-wheel odometry pods	Moderate	Good. Unpowered wheels don't slip.
goBILDA Pinpoint or SparkFun OTOS	Moderate	Good, and much simpler to mount and wire

Budget for this before you promise your team a pathing auto. It's the part that makes it feel like magic instead of feeling like a worse version of page 14.

Setting Pedro up

Repository URLs and version numbers change with every Pedro release. Get the current ones from the install page at `pedropathing.com`. Everything else on this page stays the same.

1 — Add it to your project

Easiest path, and the one we recommend: start from the Pedro **Quickstart** repository. It is the FTC SDK with Pedro and Panels already wired in and a Constants file already stubbed out. Use it as a GitHub template the same way you would use the plain SDK.

To add Pedro to a project you already have, you edit two Gradle files:

```
// build.dependencies.gradle, inside repositories { }
maven { url = "<repository URL from the install page>" }

// TeamCode/build.gradle, inside dependencies { }
implementation "com.pedropathing:core:<version>"
implementation "com.pedropathing:ftc:<version>"
implementation "com.bylazar:panels:<version>"
```

Copy the exact URL and version strings off the install page rather than out of a tutorial. They have changed more than once, and a stale version string produces a Gradle error that looks nothing like the real problem. Then press **Sync Now**.

2 — Tell it about your robot

Set your motor names and directions, and pick your localizer: drive encoders, dead wheels, Pinpoint, or OTOS. Then run the tuning OpModes **in the order the docs list them**, starting with the forward and lateral multipliers. Budget one full meeting and don't skip ahead, because later steps assume the earlier ones are correct.

Recent versions offer two following algorithms. The original tunes a set of PIDs. The newer **predictive braking** option replaces most of those PIDs with a braking model, and for most teams it is both easier to tune and faster. Check which one your version defaults to before you start.

3 — Draw the path

Open `visualizer.pedropathing.com`, drag control points on the field diagram, and copy out the generated code. Do this on the couch the night before a meeting.

4 — Write the auto

The structure is on the next page. Notice that it's the same state-machine idea as page 11. The robot never waits, it asks the follower whether it has arrived yet.

> **Before you write a path, prove the tuning.** Run the built-in straight-line and turn tests and watch them in Panels. If the robot's drawn position doesn't match the real robot, fix that first. No path will work on top of bad localization.

What you need before any of this

- Android Studio. Pedro cannot be used from OnBot Java or Blocks.
- A localizer you trust (see the table on page 16).
- A field, or at least a flat measured space. Tuning on a carpet offcut in a hallway gives you numbers that don't hold up on a real field.
- One uninterrupted meeting. Tuning half-finished is worse than not started.

A Pedro auto, start to finish

Poses are field coordinates in inches. Paths are built once at init, then the state machine walks through them without ever blocking.

```
@Autonomous(name = "Auto - Pedro", group = "Auto")
public class AutoPedro extends OpMode {

    private Follower follower;
    private PathChain scorePath, parkPath;
    private int pathState;
    private Timer stateTimer;
    private final Pose startPose = new Pose(9,  60, Math.toRadians(0));
    private final Pose scorePose = new Pose(37, 50, Math.toRadians(180));
    private final Pose parkPose  = new Pose(60, 15, Math.toRadians(90));

    public void buildPaths() {
        scorePath = follower.pathBuilder()
                .addPath(new BezierLine(startPose, scorePose))
                .setLinearHeadingInterpolation(
                        startPose.getHeading(), scorePose.getHeading())
                .build();

        parkPath = follower.pathBuilder()
                .addPath(new BezierLine(scorePose, parkPose))
                .setLinearHeadingInterpolation(
                        scorePose.getHeading(), parkPose.getHeading())
                .build();
    }

    // Same state machine idea as page 11. The robot never waits,
    // it checks whether the follower has arrived.
    public void autonomousPathUpdate() {
        switch (pathState) {
            case 0:
                follower.followPath(scorePath);
                setPathState(1);
                break;

            case 1:
                if (!follower.isBusy()) {       // arrived
                    // score here — run your mechanism
                    setPathState(2);
                }
                break;

            case 2:
                if (stateTimer.getElapsedTimeSeconds() > 0.5) {
                    follower.followPath(parkPath);
                    setPathState(3);
                }
                break;

            case 3:
                if (!follower.isBusy()) setPathState(-1);   // done
                break;
        }
    }

    public void setPathState(int state) {
        pathState = state;  stateTimer.resetTimer();
    }

    @Override public void init() {
        stateTimer = new Timer();
        follower   = Constants.createFollower(hardwareMap);
        buildPaths();
        follower.setStartingPose(startPose);
    }

    @Override public void loop() {
        follower.update();              // must be called every loop
        autonomousPathUpdate();
        telemetry.addData("path state", pathState);
        telemetry.addData("pose", follower.getPose());
        telemetry.update();
    }
}
```

Debugging: open Panels and watch the pose while it runs. If the drawn position doesn't match the real robot, the problem is localization or tuning, not your path.

AprilTags, custom models, and the Limelight 3A

Vision is rung four. If your auto isn't consistent without it, vision will not fix that. It will give you a new thing to blame.

AprilTags — start here

AprilTags are the printed square markers on the field. They're the highest-value, lowest-effort vision in FTC: detection is reliable, they're already on the field, and the SDK does the pose math for you. Point a webcam at one and you get back:

What you get	What it means	What you'd use it for
`id`	Which tag it is	Choosing between three auto routines
`range`	Distance to the tag	Driving to a fixed distance from a scoring spot
`bearing`	Angle left/right of centre	Turning until you're aimed at it
`yaw`	How rotated the tag is relative to you	Squaring up to a wall or goal

This works in **OnBot Java too**. The VisionPortal samples run there. You don't need Android Studio for AprilTags.

Your own object detection model

To recognize this season's game element, you train a model: collect a few hundred labelled photos, train it, and drop the resulting `.tflite` file into the project. It's genuinely doable and it's a real skill. Two warnings:

- **Photograph in the lighting you'll compete in,** not just your shop. A model trained under fluorescent lights in a classroom fails under gym lights every time.
- **Budget hours, not minutes.** Labelling images is the slow part and there's no shortcut.

The Limelight 3A

A small camera with its own computer inside. It does the vision itself and hands your OpMode the answers, so the Control Hub isn't doing image processing. You configure pipelines by pointing and clicking in a browser.

What it buys you

- AprilTag tracking and field localization built in
- Colour/object tracking with no code
- A browser trainer for custom models
- Far less tuning time than rolling your own

What it costs

- Money, and a mounting spot on the robot
- **One per Control Hub** — that's the limit
- Check this season's Game Manual Part 1 to confirm it's still legal before buying

> **The fix for most "it doesn't detect" problems is exposure, not code.** Lower the camera exposure until motion blur disappears. A slightly dark, sharp image beats a bright, smeared one every time.

Reading AprilTags with a webcam

This runs in Android Studio and in OnBot Java. It picks an auto routine from the tag it sees during init, which is legal and is the most common use of vision in FTC.

```
@Autonomous(name = "Tag Webcam", group = "Vision")   // or @TeleOp
public class TagWebcam extends LinearOpMode {

    @Override
    public void runOpMode() {
        AprilTagProcessor tagProcessor = new AprilTagProcessor.Builder()
                .setDrawTagID(true)
                .setDrawAxes(true)
                .build();

        VisionPortal portal = new VisionPortal.Builder()
                .setCamera(hardwareMap.get(WebcamName.class, "webcam1"))
                .setCameraResolution(new Size(640, 480))
                .addProcessor(tagProcessor)
                .build();

        // Fixed, low exposure kills motion blur. Tune on the field.
        while (!isStopRequested()
               && portal.getCameraState() != VisionPortal.CameraState.STREAMING) {
            sleep(20);
        }
        ExposureControl exposure = portal.getCameraControl(ExposureControl.class);
        exposure.setMode(ExposureControl.Mode.Manual);
        exposure.setExposure(6, TimeUnit.MILLISECONDS);
        portal.getCameraControl(GainControl.class).setGain(250);

        int targetId = -1;

        // Look for a tag BEFORE start. This is legal, and it's how
        // pick between auto routines.
        while (opModeInInit()) {
            for (AprilTagDetection d : tagProcessor.getDetections()) {
                if (d.metadata != null) targetId = d.id;
            }
            telemetry.addData("tag seen", targetId);
            telemetry.update();
        }

        waitForStart();
        if (isStopRequested()) return;

        if (targetId == 1)      { /* routine A */ }
        else if (targetId == 2) { /* routine B */ }
        else                    { /* fallback — always have one */ }

        while (opModeIsActive()) {
            for (AprilTagDetection d : tagProcessor.getDetections()) {
                if (d.metadata == null) continue;
                telemetry.addData("id",      d.id);
                telemetry.addData("range",   "%.1f in", d.ftcPose.range);
                telemetry.addData("bearing", "%.1f deg", d.ftcPose.bearing);
                telemetry.addData("yaw",     "%.1f deg", d.ftcPose.yaw);
            }
            telemetry.update();
        }
        portal.close();
    }
}
```

Always write the fallback. A vision auto that does nothing when it can't see is worse than a time-based auto that always parks. If no tag is detected, run a default routine. Never just stand still.

The same thing with a Limelight 3A

The Limelight does the vision itself, so your OpMode just asks it for the answer. Pipelines are built by pointing and clicking in a browser, not in code.

```
@TeleOp(name = "Tag Limelight", group = "Vision")
public class TagLimelight extends LinearOpMode {

    @Override
    public void runOpMode() {
        Limelight3A limelight = hardwareMap.get(Limelight3A.class, "limelight");

        DcMotor frontLeft  = hardwareMap.get(DcMotor.class, "frontLeft");
        DcMotor backLeft   = hardwareMap.get(DcMotor.class, "backLeft");
        DcMotor frontRight = hardwareMap.get(DcMotor.class, "frontRight");
        DcMotor backRight  = hardwareMap.get(DcMotor.class, "backRight");
        frontLeft.setDirection(DcMotor.Direction.REVERSE);
        backLeft .setDirection(DcMotor.Direction.REVERSE);

        telemetry.setMsTransmissionInterval(11);
        limelight.pipelineSwitch(0);    // pipeline you built in the browser
        limelight.start();

        waitForStart();

        while (opModeIsActive()) {
            LLResult result = limelight.getLatestResult();

            if (result != null && result.isValid()) {
                telemetry.addData("tx", result.getTx());   // left/right offset
                telemetry.addData("ty", result.getTy());   // up/down offset
                telemetry.addData("pose", result.getBotpose());

                // Simple auto-aim: turn until the target is centred.
                // Motors declared as on page 9. Add a deadband or it hunts.
                double turn = result.getTx() * 0.02;
                frontLeft.setPower( turn);  backLeft .setPower( turn);
                frontRight.setPower(-turn); backRight.setPower(-turn);
            } else {
                telemetry.addLine("no target");
            }
            telemetry.update();
        }
        limelight.stop();
    }
}
```

Worth knowing before you buy one

- One Limelight per Control Hub. That's a hard limit.
- Confirm it's still legal in this season's Game Manual Part 1 before spending the money.
- Save your pipelines before a firmware update. Updating wipes them.
- It still needs a fallback routine, exactly like the webcam version.

Habits that save seasons

None of this is about writing better code. All of it is about not losing a season to something avoidable.

Use Git, starting today

A free GitHub account, a repository for your team, and a commit at the end of every meeting. When two people program, one branch each. The most common way FTC teams lose an entire autonomous is a laptop that dies in February with the only copy on it.

Telemetry is your debugger

You can't attach a debugger to a robot driving around a field, so print what you need to see: the current state name, encoder positions, whether a sensor sees anything, and your loop time. If you're guessing at what the robot is doing, you aren't printing enough.

Watch your loop time

Print how many milliseconds each pass takes. If it climbs past about 20 ms, every control on the robot starts feeling mushy. The usual culprit is reading the same sensor repeatedly. Cache your hub reads once per loop and reuse the values.

Name things once

One naming convention for config entries, used everywhere, written on tape on the robot. Half of all "the code is broken" reports are a spelling difference.

Don't write code at competition

Bring the laptop, and bring two if you can. But plan to test at home. Twenty minutes between matches is not a development window, and every team that tries to write a new auto in the pit ends up with a robot that does nothing in the next match.

Write down why, while you remember

Your engineering portfolio and the Control Award both want your software decisions documented: what you tried, what failed, and why you chose what you chose. Writing it in October takes five minutes a meeting. Reconstructing it in February takes a weekend.

End-of-meeting checklist

- [ ] Code committed and pushed
- [ ] Config file matches the robot, and the tape labels still match the config
- [ ] Batteries on the charger
- [ ] One line in the notebook: what we changed, and what we'll test next

Your first four weeks

Write real dates in. A plan without dates is a wish, and week four arrives whether you planned for it or not.

Week 1 — by _____

- ☐ Somebody has a working dev environment (Android Studio or OnBot Java)
- ☐ Control Hub configured, default password changed, names written on tape
- ☐ Robot drives in TeleOp — page 9
- ☐ Code is in a Git repository — page 6

Week 2 — by _____

- ☐ The auto worksheet on page 15 is filled in with the whole team
- ☐ A time-based auto that leaves and parks — page 13
- ☐ Run it ten times in a row. Ten out of ten, or keep tuning.
- ☐ Physical alignment reference built so the robot starts identically every time

Week 3 — by _____

- ☐ One scoring action added to the auto
- ☐ First mechanism running as a state machine, not with sleep() — page 11
- ☐ Driver practice has started. Hours behind the sticks score more points than any library.

Week 4 — by _____

- ☐ Auto converted to encoders with heading hold — page 14
- ☐ Telemetry shows state, positions and loop time on the Driver Station
- ☐ Decision made and written down: are we doing pathing this season? Who owns it?

After that: pathing (page 16) and vision (page 19), in that order, and only once everything above is boring. Chasing rung three before rung one is repeatable is the most common way a season gets away from a team.

Where to go when you're stuck

Official documentation

Site	Use it for
`ftc-docs.firstinspires.org`	The official manual for everything: hub setup, Blocks, OnBot Java, VisionPortal, sample walkthroughs
`github.com/FIRST-Tech-Challenge/FtcRobotController`	The SDK itself, the release notes, and the sample OpModes
`firstinspires.org` → Game & Season	Game Manual Part 1 and 2. Read Part 1 for scoring and legal hardware.
`ftc-community.firstinspires.org`	Official Q&A forum, answered by people who know

Libraries and tools

Site	Use it for
`pedropathing.com`	Pedro Pathing docs, setup, and tuning guide
`visualizer.pedropathing.com`	Draw paths in your browser, copy out the code
`nextftc.dev`	Command-based framework, with a Pedro integration
`docs.limelightvision.io`	Limelight 3A setup, pipelines, model training
`learnroadrunner.com`	Road Runner, if that's the path you take

How to ask a question that gets answered

1. Say what you expected to happen and what actually happened.
2. Paste the **whole** error, not a summary of it.
3. Say what you already tried.
4. Include your setup: Android Studio or OnBot, which library, which sensor.

Do this and you'll usually have an answer in an hour. "My auto doesn't work, help" gets nothing, from anyone, ever.

Three things to remember from today

1. **Start where you can start.** OnBot Java today beats Android Studio in three weeks.
2. **Never block the loop.** Check, don't wait.
3. **Write the auto now.** A ten-point auto that works every match is worth more than a forty-point auto that works twice.

Notes